[MicroServiceLab]
MicroServices

Introduction: In this assignment, we implement a MicroService for the Asteroid game.

Objectives: The objective of this assignment is to provide sufficient information to get started with MicroServices in combination with a component framework.

Classwork:

- Create a maven module for a scoring system based on this guide [SpringMicroService].
  For inspiration see [MicroServices] branch from GitHub.
- Integrate the Scoring MicroService in the AsteroidsGame using the Spring RestTemplate, see [SpringRestTemplate].

[TestLab]
Testing Component-based Software

Introduction: In this assignment, we apply unit and integration tests for the Asteroid game.

Objectives: The objective of this assignment is to provide sufficient information to get started with testing in combination with a component framework.

Classwork:

- Read [JUnit Getting started]. If you have the need to stub or mock out dependencies to create your test then i can recommend [Mockito].
- Write a unit test for one of the components. For example, write a test for moving the player ship or a test for collision detection.

[JPMSLab3]
Java Platform Module System
Layers and Split Packages

Introduction: In this assignment, we apply the Java Platform Module System and resolve issues regarding Split Packages.

Objectives: The objective of this assignment is to get familiar with the Java Platform Moudle System (JPMS) to resolve problems regarding split packages and provide sufficient information to get started with component-oriented design and assembly practice using JPMS.

Classwork:

- Try to rename two classes from two different modules to the same unique class name and see what happens.
- Resolve the issue of the introduced split packages by implementing a JPMS Module Layer. You can see an Hello World example in the GitHub repo [Split Packages and JPMS layers].
- Create a `plugins` folder in your project `root` folder and move one of the split package modules to that folder, see GitHub Branch [JPMSModuleLayers] for inspiration on how to utilize the ModuleLayer API with the JDK ServiceLoader.

[JPMSLab2]
Java Platform Module System Services
Asteroids Game

Introduction: In this assignment, we apply the Java Platform Module System Services (JPMS) in the Asteroid game setting.

Objectives: The objective of this assignment is to get familiar with the Java Platform Moudle System Services (JPMS) and provide sufficient information to get started with component-oriented design and assembly practice using JPMS.

Classwork:

- Continue to implement your Asteroids game using the Java Platform Module System Services. Please use the [AsteroidsFX] solution example for inspiration only.
- Declare `provides`, `with` and `uses` in `module-info.java` files for relevant modules in the Asteroids game.

[JPMSLab1]
Java Platform Module System
Asteroids Game

Introduction: In this assignment, we apply the Java Platform Module System (JPMS) in the Asteroid game setting.

Objectives: The objective of this assignment is to get familiar with the Java Platform Moudle System (JPMS) and provide sufficient information to get started with component-oriented design and assembly practice using JPMS.

Classwork:

- Continue to implement your own Asteroids game using the Java Platform Module System (JPMS). The solutions example can be found here [AsteroidsFX]. Please only use the solution example as inspiration (don't copy paste).
- Declare imports and exports in `module-info.java` files for each module in the Asteroids game. You can see an example in the Player module ect.

Resources:

- [Java Modules in Real Life]
- [Modularity with Oliver Drotbohm]
- [Java Modules Cheat Sheet]
- [The Java Module System with Sander Mak]
- [Code-First Java Module System Tutorial]
- [Five Command Line Options To Hack The Java Module System]
- [The Java Module System Beyond The Basics]
- [Escaping The Jar Hell With Jigsaw Layers]

[JavaLab]
Java Service Loader

Introduction: In this project, the simple Asteroids game utilizes the *ServiceLoader* available out of box in Java platform Standard Edition.

Objectives: The goal of this assignment is to: 1) to understand the pros and cons of the basic ServiceLoader extension mechanism provided by the Java standard platform, 2) to understand the primary reason for introducing component interfaces, and 3) what a whiteboard component model looks like and how to use it.

Classwork:

- Follow the steps in [Creating Extensible Applications] and try out [The dictionary example] to get acquainted with the Java ServiceLoader mechanism. This part of the lab is optional and only for extra tutorials.
- Automate the Component Assembly in your own Asteroids game using the build-in *ServiceLoader* in Java (whiteboard component model).
  Please use the *ServiceLoader* helper class is provided in here in project [AsteroidsServiceLoader]for inspiration only.

Resources:

- [Java 7 SE ServiceLoader API]